Secure Computing

Supervisor: Dr. Chatura Seneviratne
Co-Supervisor 1: Prof. Dr. An Braeken
Co-Supervisor 2: Mr. Pramitha Fernando

EG/2021/4474 Dewmith M.K.J.
EG/2021/4485 Dilshan M.D.K.
EG/2021/4807 Sewinda L.L.D.
EG/2021/4832 Thilohith T.K.

02/01/2026

CONTENT

01 | Hardware Roots of Trust & TEEs

02 | The Mechanics of Attestation

03 | Software Identity & Advanced Verification

04 | Relevance to Our FYP

PART 01

Hardware Roots of Trust & TEEs

Hardware Root of Trust

If there is no trust on hardware, then there can be no trust in the system at all

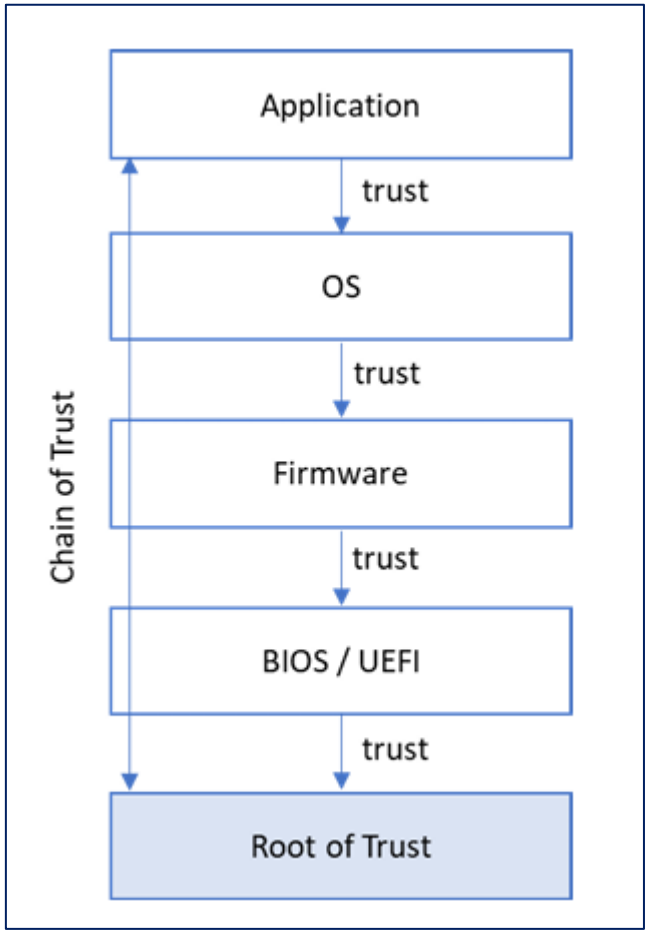


Figure 01: Root of Trust [3]

If the **Operating System (OS)** is compromised (e.g., by a rootkit or malicious administrator), it can manipulate or inspect everything running above it.

Software cannot validly attest to its own integrity because a compromised OS can simply "lie" to the security tools. Therefore, security must be anchored in something immutable—**Hardware**.

This effectively forms a Chain of Trust throughout the stack from the application to the Root of Trust residing on hardware

Trusted Computing Group has defined Trusted Computing Base (TCB) that is a collection of system resources consisting of hardware and software that is responsible for maintaining the security policy of a system.

TPM (Trusted Platform Module)

Trusted Platform Module (TPM) is a **cryptographic solution** defined by Trusted Computing Group (TCG), which allows **securely create and store cryptographic keys, secrets, and information**

- **Validate OS and the Firmware** on a host are as expected and that they have not been tampered
- TPM is typically implemented and deployed as a separate and independent component **(chip).**
- **TPM 2.0** specification supports the processor manufacturers to embed the **TPM capabilities inside** their chipsets without the need to have an external component
- The TPM standard defines a hardware root of trust **(HRoT)** which has been widely accepted more secure than software-based solutions.

The functionalities offered by TPM,

- Symmetric encryption
- Random number generation cryptographic services
- Protected persistent storage for small amounts of data
- Counters and extendible registers
- Protected pseudo-persistent store for keys and data
- Platform identities
- Attestation

Use of TPM for Open RAN

Open RAN, the containerized components are **deployed on cloud hosts** that may be owned and managed either by **the operator of the Open RAN system** or **external cloud service provider**

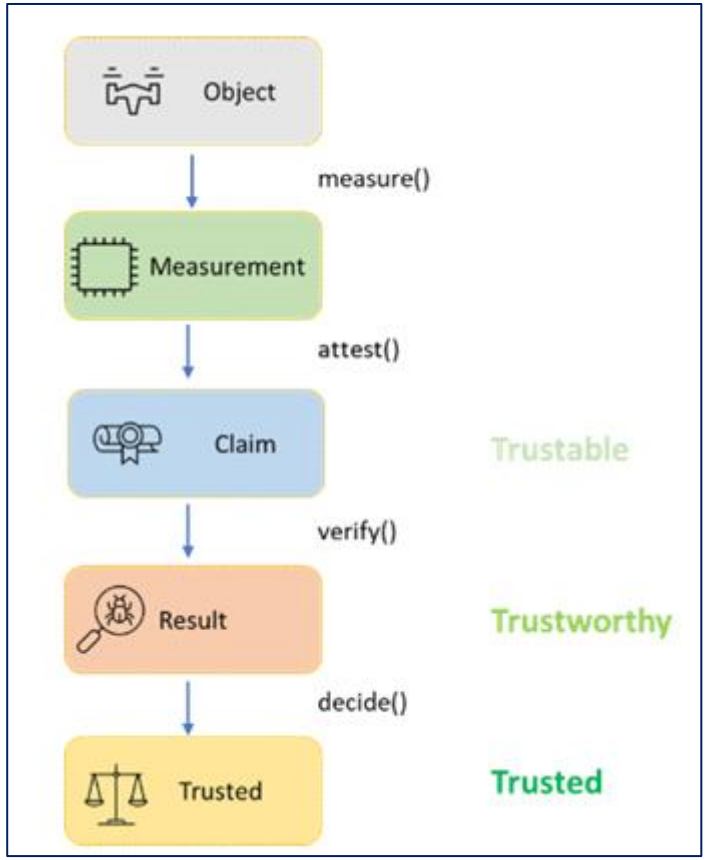


Figure 02-I : Remote attestation and levels of trust [3]

- ❑ It is required that there is **trust on whole system** including all software and hardware components.
- ❑ Remote attestation supported by **TPM** can be used to reliably verify and decide on trust of a system and its components
- ❑ TPM provides **3** Root of Trusts that fulfill the above requirements:
  - Root of Trust for Measurement (RTM) – initial integrity measurement
  - Root of Trust for Storage (RTS) – safely storing
  - Root of Trust for Reporting (RTR). – authenticity and non-repudiation

Trusted Execution Environments (TEEs)

TEE as an environment that enforces that any code **within the environment cannot be tampered** with, and any data used by such code cannot be read or tampered with by any code outside the environment.

- A TEE is a **software entity**—either **a thread** with some memory or **a virtual machine**—that the processor promises to protect.
- **Specifically, the processor will encrypt the TEE's memory before evicting it, and decrypt it only to return it to the same TEE.**
- An attestation for a TEE is a digital signature or Message Authentication Code that asserts that a TEE **E** is under the control of particular code **C**, and associated with other data **D**.
- Attestations, require support from the **processor that must guarantee the TEE**. As we use TEEs, the data **D** always includes a public key **K**. The TEE generates the key pair **K** K^{-1}**, on startup**, and protects the **private part** K^{-1}, **inserting K into D**. Thus, any remote entity that obtains an attestation for E,C,K,... can use **K to create secure channels to E**. Messages over these channels are **entrusted to the code C**. If the **code C faithfully implements a protocol X**, **then E uses the private key** K^{-1} **only in accordance with that protocol X.**

Intel SGX (Software Guard Extensions)

Intel SGX is a **specific TEE implementation** that allows an application to instantiate a protected region of memory called an **"Enclave."**

❑ **How it works:**

The hardware isolates the enclave's memory from the rest of the system. Even if malware gains full administrator (root) privileges on the OS, it cannot read the enclave's memory.

Intel's SGX attestation mechanism involves four elements.

1. **Local quote** : a subject enclave **σ** can be verified by a target enclave **τ** resident on the same processor
2. To obtain attestations for entities on other devices, **a remote quote** is made by a particular enclave, the quoting enclave $\boldsymbol{\tau q}$.
3. **Intel's attestation server** validates remote quotes.
4. **The attestation client queries** the server over TLS

Intel TDX (Trust Domain Extensions)

While SGX protects small *parts* of an application (enclaves), **Intel TDX** protects entire **Virtual Machines (VMs)**. It creates a "Trust Domain" (TD)—a hardware-isolated VM that is invisible to the cloud provider's Hypervisor and other VMs.

- **Key Difference from SGX:**

SGX requires rewriting applications to split them into "trusted" and "untrusted" parts. TDX allows you **to take an entire existing OS and run it securely without code changes**.

- **Mechanism:**

It uses **MKTME** (Multi-Key Total Memory Encryption) to encrypt the VM's memory with a unique key. **[4]**

PART 02

The Mechanics of Attestation

Introduction to Attestation

Definition: Attestation is the activity of making a claim about a system's properties by supplying cryptographic evidence to an appraiser.

- **Lying Endpoint Problem:** A compromised OS can intercept system calls and report "All System Green". Software cannot validly attest to itself.
- **The Hardware Solution:**
  - **Root of Trust (RoT):** A computing engine that must be implicitly trusted (e.g., TPM)
  - **Chain of Trust:** The recursive process where the RoT measures the next component before handing over control.
- **Key Actors**
  - **Prover (Attester):** The target system providing evidence of its integrity
  - **Verifier (Appraiser):** The remote entity that evaluates the evidence against a policy
- **The Shift in Philosophy**
  - **Traditional:** Trusting the OS and kernel implicitly
  - **Modern:** Assuming the environment is untrusted until proven by Hardware Rooted Evidence

The Measurement Chain (CRTM & PCRs)

- **CRTM (Core Root of Trust for Measurement):** The immutable boot code (BIOS block) that executes first.
- **Transitive Trust:** CRTM measures BIOS → BIOS measures Bootloader → Bootloader measures Kernel.
- **Hashing:** The system creates an SHA-256 "fingerprint" of code, configuration and data before it executes.
- **The TPM & PCRs:** Measurements are stored in Platform Configuration Registers (PCRs).
- **The "Extend" Operation:**
  - PCR(new) = SHA256 (PCR(old) || SHA256(Data))
  - PCRs are Append-Only; they cannot be overwritten or deleted
- **Static vs. Dynamic:**
  - **Static (SRTM):** Measured boot (BIOS → Kernel)
  - **Dynamic (DRTM):** Runtime verification (e.g., Intel SGX)

The Remote Attestation Workflow

- **The Challenge:** Verifier sends a **Nonce** (160bit+ random number) to the Prover.
- **The Quote (Evidence):** TPM signs the (PCR values || Nonce) using the private Attestation Identity Key (AIK).
- **Key Management (Privacy):**
  - **Endorsement Key (EK):** Unique, permanent hardware ID (Burned in factory)
  - **Attestation Identity Key (AIK):** An alias key used for signing quotes to prevent hardware tracking (Privacy Preserving)
- **Golden Measurements:**
  - **RIM (Reference Integrity Manifest):** The database of known-good hash values provided by software vendors.
- **The Appraisal (Verification):**
  1. Verifier validates the TPM's signature with a public key.
  2. Verifier checks the Nonce to prevent Replay Attacks.
  3. Verifier compares hashes against 'Golden Measurements'

Software & Workload Attestation (SPIRE/SPIFFE)

- **The Gap:** TPM measures the Host, but not containerized applications
- **SPIRE (SPIFFE Runtime Environment):**
  - Node Attestation: Uses TPM/Cloud API to verify the underlying VM/Server
  - Workload Attestation: Introspects the Kernel (Unix Domain Sockets, cgroups) to identify processes.
- **SVID (SPIFFE Verifiable Identity Document):**
  - A short-lived X.509 certificate issued only if attestation passes.
  - Used for mTLS authentication between microservices (xApps).
- **Binary vs. Semantic:** Moving from “ Is this the correct binary?” to “ Does this process have the right attributes?”

PART 03

Software Identity & Advanced Verification

Why Software Identity Matters

Modern distributed systems no longer operate on fixed, trusted machines.

- Applications are deployed as microservices and containers
- Services frequently scale, migrate, and restart
- Network identifiers (IP, hostname) are dynamic and unreliable

Traditional security relied on hardware or machine identity, but this is insufficient in cloud-native environments.

Security must shift from *"where the code runs"* to *"what code is running."*
Each workload must prove its identity independently of the infrastructure.

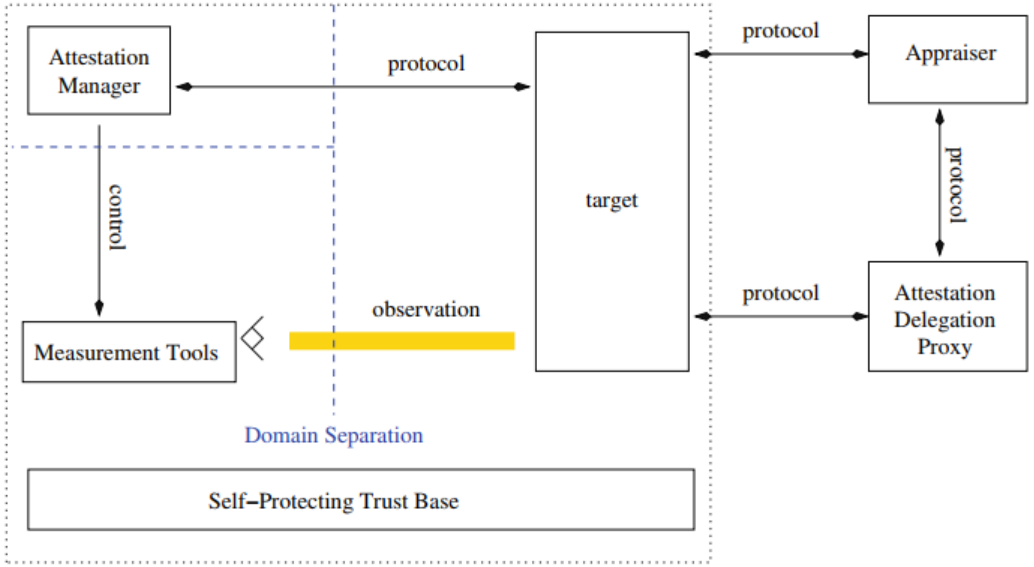


Figure 03: Remote attestation in distributed environments

SPIFFE, SPIRE, and SVIDs

- **SPIFFE** defines a standard identity format for software workloads
- **SPIRE** is a production-grade implementation of SPIFFE
- A workload receives a **SPIFFE Verifiable Identity Document** (SVID)

An SVID is:

  - Cryptographically verifiable
  - Short-lived (reduces replay and theft risk)
  - Used for mutual authentication between services

Example SPIFFE ID:

```
spiffe://example.org/payment-service
```

- Identity is automatically issued, not manually configured.

How Software Identity Is Issued (Attestation Flow)

SPIRE issues identities only after **successful attestation**.

1. **Node Attestation**
   Verifies the host (VM, node, or physical machine)
   Ensures a trusted execution environment
2. **Workload Attestation**
   Verifies the specific process or container
   Confirms expected runtime properties

Only when both succeed, an SVID is issued.

This prevents:

- Unauthorized workloads
- Credential reuse
- Rogue services impersonation

Types of Software Attestation

Software attestation validates trustworthiness before identity issuance.

Attestation Type	What It Verifies	Characteristics
Measurement-Based	Code integrity	Strong guarantees, rigid
Context-Based	Runtime environment	Flexible, update-friendly

- Measurement-based attestation uses cryptographic hashes
- Context-based attestation checks runtime metadata

Containerized systems commonly use context-based attestation for flexibility.

Limitations of Traditional Attestation

Traditional attestation assumes:

- Trusted hardware
- Trusted OS or hypervisor

However:

- If the OS or hypervisor is compromised, attestation results are unreliable
- It proves platform state, not computation correctness

Traditional attestation cannot verify:

- What computation was performed
- Whether outputs are correct

zkVM: Verifiable Computation

A **Zero-Knowledge Virtual Machine (zkVM)** executes programs and produces a cryptographic proof of execution.

The proof guarantees:

- Correctness – computation followed the rules
- Integrity – output cannot be forged
- Privacy – inputs remain hidden

Trust shifts from hardware to mathematical proof.

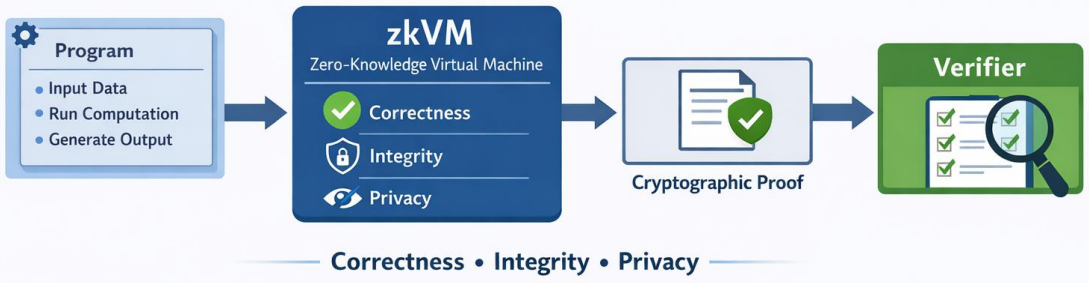


Figure 04: Verifiable computation workflow using a zkVM

zkVM vs Traditional Attestation

Aspect	Traditional Attestation	zkVM
Trust Base	Hardware & OS	Cryptography
Verifies	Platform state	Computation
Privacy	Limited	Strong

- Traditional attestation answers **“who ran the code”**
- zkVM answers **“what computation was performed correctly”**

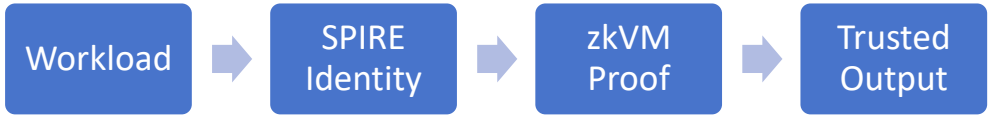
zkVMs remove the need to trust the execution platform by shifting verification to cryptographic proofs.

Combining SPIRE and zkVM

- **SPIRE** verifies *who* the workload is
- **zkVM** verifies *what* the workload computed

Together they provide:

- Identity assurance
- Computation correctness
- End-to-end trust



This combination enables **zero-trust, proof-based systems**.

Key Takeaways

- Modern systems require software-based identity
- SPIRE enables automated, workload-level trust
- Traditional attestation is not sufficient
- zkVMs introduce verifiable computation

Security evolves from **trust assumptions** to **cryptographic proof**

PART 04

Threat Containment for xApps

Remote Attestation – Proactive Integrity

- The Problem: Current manual hashing (used by seniors) is slow and not scalable for real-time O-RAN.
- The Proposal: Implement Remote Attestation for proactive integrity.
- How it works: The RIC performs a hardware-based measurement of an xApp's code before it starts.
- Impact: If code is modified (e.g., to include a "Flood Attack"), the measurement fails, and the xApp is blocked from connecting.

Intel SGX Enclaves (Protecting the Watcher)

- The Problem: If an xApp is compromised, the attacker will immediately try to disable the security "Watcher" script.
- The Strategy: Run the containment script inside an Intel SGX Enclave.
- How it works: SGX creates a hardware-encrypted "safe" in the CPU that even the OS cannot access.
- Impact: The security monitor becomes untouchable; it cannot be killed or modified by a malicious xApp.

SPIRE (Identity-Based Containment)

- The Problem: Static credentials can be stolen. We need a way to "silence" a compromised app.
- The Strategy: Use SPIRE to issue dynamic identities (SPIFFE IDs).
- How it works: An xApp must have a valid certificate to talk to the Near-RT RIC. If an app fails its integrity check, SPIRE revokes the certificate.
- Impact: The compromised xApp is effectively "contained"—it stays running but can no longer send commands to the radio (E2 interface).

zkVM (Verifiable Monitoring)

- The Problem: Sharing raw security logs with the SMO (supervisor layer) is a privacy risk and inefficient.
- The Strategy : Use zkVM (Zero-Knowledge Virtual Machine) for reporting.
- How it works: zkVM runs the containment logic and produces a mathematical proof that the xApp was killed.
- Impact: Provides a "Proof of Security" to the management layer without sharing sensitive raw network data.

References

1. Understanding Attestation: Analyzing Protocols that use Quotes, Joshua D. Guttman and John D. Ramsdell, The MITRE Corporation
2. Principles of remote attestation George Coker · Joshua Guttman · Peter Loscocco · AmyHerzog · Jonathan Millen · Brian O'Hanlon · John Ramsdell · Ariel Segall · Justin Sheehy · Brian Sniffen
3. Security and trust in Open RAN Master's thesis Information Technology Cybersecurity 2024 Pekka Kuure Commissioned by Nokia Bell Labs
4. "Intel Trust Domain Extensions (Intel TDX) Module Base Architecture Specification," 2024.

THANK YOU!